

Bônus — BFF, GraphQL & Segurança

"Como integrar mobile a microsserviços sem virar cliente burro?"

Prof. Jackson Smith Moisés Matias

PUC Minas IEC · conteúdo gravado, sem entrega/pontos

Recap — PWA Offline-First (Aula 4)

Aula 4 foi cancelada (24/06). Conteúdo disponível nos slides e leituras.

- Service Worker lifecycle e cache strategies
- IndexedDB + Background Sync para offline-first
- Lighthouse CI bloqueando merge

Atividade 4 — prazo foi 23/06. Bot ainda aceita PR tardio até hoje à meia-noite.

Agenda da aula (3h30)

1. **Bloco 1 (75min)** — BFF, GraphQL, Apollo Client
2. **Intervalo (30min)**
3. **Bloco 2 (75min)** — Hands-on: integração GraphQL + OAuth/PKCE + SSL pinning
4. **Esquenta (15min)** — Atividade 5 + Quiz IA

Objetivos de aprendizagem

- **Aplicar** padrão BFF para apps mobile em arquiteturas distribuídas
- **Implementar** GraphQL client (Apollo) com cache normalizado em RN
- **Configurar** OAuth 2.1 + OIDC + PKCE corretamente
- **Aplicar** OWASP Mobile Top 10 (2024) ao app
- **Implementar** SSL pinning testado contra MITM real (mitmproxy)

O problema da integração mobile com microsserviços

Cenário típico em 2026:

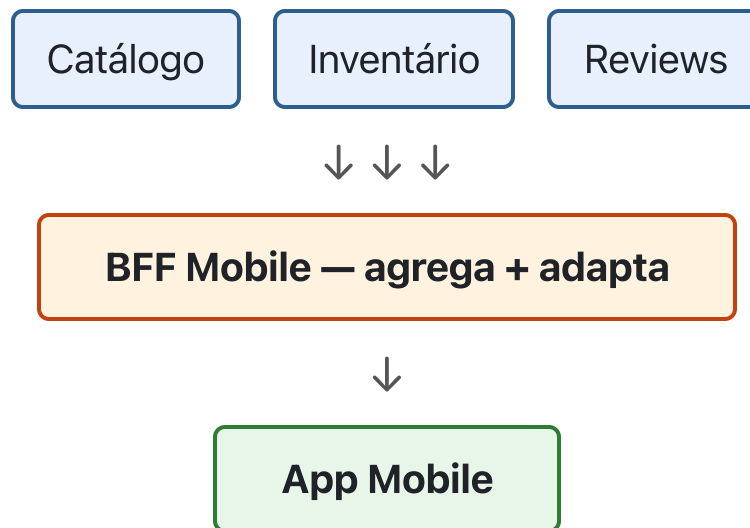
- App mobile precisa exibir tela de detalhes do produto
- Backend tem 4 microsserviços: catálogo, inventário, reviews, recomendações
- Cada um tem seu contrato REST próprio

Sem BFF: app faz 4 chamadas HTTP em rede 4G, drena bateria, 4x latência.

Com BFF: 1 chamada agregadora, BFF orquestra, app recebe payload otimizado.

Backend for Frontend (BFF)

Padrão popularizado por **Sam Newman** em *Building Microservices*.



Cada cliente (mobile, web, smart TV) tem seu próprio BFF.

Por que BFF?

1. **Reduzir round-trips** — agregar múltiplas chamadas em uma só
2. **Adaptar formato** — payload otimizado por cliente (mobile recebe menos campos que web)
3. **Isolar mudanças** — atualizar BFF sem mexer em microsserviços compartilhados
4. **Auth centralizado** — um lugar pra trocar OAuth por sessão interna
5. **Caching inteligente** — BFF pode cachear ao invés de mobile fazer

API Gateway vs BFF

API Gateway: ponto único pra TODOS os clientes (rate limit, auth, transformações genéricas).

BFF: especializado por cliente (mobile, web, smartTV).

| Coexistem. Gateway na borda; BFF logo atrás dele para cada cliente.

Tools comuns:

- API Gateway: Kong, AWS API Gateway, Apollo Router
- BFF: serviço normal Node.js / Go / Kotlin

GraphQL — paradigma alternativo

REST é orientado a recursos. **GraphQL é orientado a queries declarativas.**

```
query MovieDetail($id: ID!) {  
  movie(id: $id) {  
    id  
    title  
    poster  
    director { id name }  
    cast(limit: 5) { id name role }  
    reviews(limit: 10) { id rating comment user { name } }  
  }  
}
```

Uma chamada, exatamente os dados que o cliente precisa.

GraphQL — pontos fortes

- **Cliente decide** o shape do payload (sem over-fetch / under-fetch)
- **Schema strong-typed** — geração de tipos TS automática
- **Single endpoint** — uma URL, várias queries
- **Federation v2** — múltiplos serviços expostos como schema unificado
- **Tooling forte** — Apollo Studio, GraphiQL, codegen

GraphQL — armadilhas

- ✗ **N+1 query problem** — cada campo pode disparar nova chamada DB no resolver
→ resolver com **DataLoader** (batching + caching)
- ✗ **Queries sem profundidade limitada** — atacante pode pedir queries gigantescas
→ query depth limiting + rate limit
- ✗ **Caching mais complexo que REST** — não é HTTP cacheable trivialmente
- ✗ **Curva de aprendizado** — schema design, resolvers, federation

Apollo Client em RN

```
import { ApolloClient, InMemoryCache, ApolloProvider } from '@apollo/client';
import { setContext } from '@apollo/client/link/context';
import { createHttpLink } from '@apollo/client';

const httpLink = createHttpLink({ uri: 'https://bff.myapp.com/graphql' });

const authLink = setContext(async (_, { headers }) => {
  const token = await getToken();
  return {
    headers: {
      ...headers,
      authorization: token ? `Bearer ${token}` : '',
    },
  };
});

const client = new ApolloClient({
  link: authLink.concat(httpLink),
  cache: new InMemoryCache({
    typePolicies: {
      Movie: { keyFields: ['id'] },
      User: { keyFields: ['id'] },
    },
  }),
});
```

Apollo — cache normalizado

```
import { useQuery, gql } from '@apollo/client';

const GET_MOVIE = gql`
  query GetMovie($id: ID!) {
    movie(id: $id) {
      id
      title
      director { id name }
    }
  }
`;

function MovieScreen({ id }) {
  const { data, loading, error } = useQuery(GET_MOVIE, { variables: { id } });
  // ...
}
```

Quando uma mutation atualiza Movie:42, **todas as telas** que usam Movie:42 re-renderizam automaticamente. Cache normalizado por chave (`__typename: id`).

OAuth 2.1 + OIDC + PKCE em mobile

OAuth 2.1 consolidou best practices. Em mobile, **PKCE é obrigatório** (RFC 7636).

```
App → Auth Server // 1. /authorize?code_challenge=hash(verifier)
App ← Auth Server // 2. login via WebView
App ← Auth Server // 3. authorization_code
App → Auth Server // 4. /token {code + verifier} ← prova posse
App ← Auth Server // 5. access_token + refresh_token
```

Por que PKCE?

Sem PKCE, app malicioso instalado no device pode interceptar o `authorization code` (via custom URL scheme registrado igual) e trocá-lo por tokens.

Com PKCE: app gera `code_verifier` aleatório, manda hash (`code_challenge`) na requisição inicial. Para trocar code por token, precisa enviar o `code_verifier` original. Atacante interceptou o code mas não tem o verifier → falha.

PKCE não é opcional em mobile. É contramedida específica.

Implementação OIDC + PKCE em RN

```
import { authorize } from 'react-native-app-auth';

const config = {
  issuer: 'https://auth.myapp.com',
  clientId: 'mobile-app',
  redirectUrl: 'com.myapp:/oauth2callback',
  scopes: ['openid', 'profile', 'offline_access'],
  usePKCE: true, // padrão em react-native-app-auth, mas explicitar
};

async function login() {
  try {
    const result = await authorize(config);
    await SecureStorage.setItem('access_token', result.accessToken);
    await SecureStorage.setItem('refresh_token', result.refreshToken);
    return result;
  } catch (error) {
    console.error('Login failed', error);
  }
}
```

Secure storage — Keychain / Keystore

iOS: Keychain Services

Android: Keystore + EncryptedSharedPreferences

```
import * as Keychain from 'react-native-keychain';

// Salvar
await Keychain.setGenericPassword('user', JSON.stringify({ accessToken, refreshToken }));

// Recuperar
const credentials = await Keychain.getGenericPassword();
if (credentials) {
  const { accessToken, refreshToken } = JSON.parse(credentials.password);
}

// Limpar
await Keychain.resetGenericPassword();
```

Nunca armazene tokens em `AsyncStorage` ou `localStorage` — texto puro e extraível.

OWASP Mobile Top 10 (2024)

1. **M1** — Improper Credential Usage
2. **M2** — Inadequate Supply Chain Security
3. **M3** — Insecure Authentication/Authorization
4. **M4** — Insufficient Input/Output Validation
5. **M5** — Insecure Communication
6. **M6** — Inadequate Privacy Controls
7. **M7** — Insufficient Binary Protections
8. **M8** — Security Misconfiguration
9. **M9** — Insecure Data Storage
10. **M10** — Insufficient Cryptography

| Tabela mudou de 2016 → 2024. Verificar versão atual sempre.

SSL pinning

Sem pinning, atacante com proxy + CA root instalado no device (Charles, mitmproxy) intercepta tráfego TLS legitimamente.

Com SSL pinning: app só aceita certificados específicos pré-configurados.

```
import { fetch } from 'react-native-ssl-pinning';

const result = await fetch('https://bff.myapp.com/graphql', {
  method: 'POST',
  body: JSON.stringify(query),
  pkPinning: true,
  sslPinning: {
    certs: ['cert-prod-2026', 'cert-prod-2026-backup'],
  },
});
```

Mantenha **2+ pins** (atual + backup) para rotação sem broken release.

Demo ao vivo — MITM com mitmproxy

1. Configurar mitmproxy no PC (mesmo Wi-Fi)
2. Instalar CA root do mitmproxy no device
3. App **sem pinning** → tráfego visível em texto puro no proxy
4. App **com pinning** → conexão recusada com erro TLS

| Demo memorável. Aluno vê a vulnerabilidade ao vivo.

Intervalo — 30min

Volta às :__

Aproveita pra:

- Esticar perna
- Subir BFF local (Docker compose) — leva ~3min cold start
- Tomar café / água

Próximo bloco é GraphQL + OAuth + SSL pinning. BFF rodando = não trava no início.

Hands-on — vamos fazer juntos (75min)

Roteiro

1. Abrir o app do curso (`02-app-rn-navegacao-estado`) — já tem RN configurado (5min)
2. Subir Apollo Server local com schema TMDB-like via `npx apollo-server` (15min)
3. Configurar Apollo Client com auth header (10min)
4. Implementar OIDC + PKCE com `react-native-app-auth` (20min)
5. Cache normalizado funcionando — mutation atualiza todas as telas (10min)
6. Demo SSL pinning + mitmproxy ao vivo (15min)

Pitfalls comuns

- ✗ OAuth implicit flow em mobile — deprecated; usar code+PKCE sempre
- ✗ Refresh token sem rotation — token roubado vale forever
- ✗ AsyncStorage para tokens — texto puro extraível
- ✗ N+1 GraphQL sem DataLoader — backend morre
- ✗ SSL pinning com 1 cert só — rotação quebra app
- ✗ Não usar state parameter no OAuth — vulnerável a CSRF

Atividade 5: Integração Segura (10 pts)

Entrega: até 07/07 (antes da Aula 6).

Entregar:

1. App RN com **GraphQL queries + mutations + cache normalizado** funcionando
2. **OIDC + PKCE** corretos (sem implicit flow)
3. **SSL pinning** configurado (testar contra mitmproxy)
4. **Relatório 2pgs** cobrindo 3 itens do OWASP Mobile Top 10 mitigados

Quiz IA Aplicada a Mobile (10 pts)

Disponível no Canvas após esta aula. Entrega: até 07/07.

- Banco de 25 questões → sorteia 10
- 1 tentativa, 30min
- Tópicos:
 - Trade-offs on-device vs cloud
 - Quantização (int4, int8, fp16)
 - Prompt engineering para mobile
 - MLC-LLM, llama.cpp, Core ML, TFLite, ONNX
 - Apple Foundation Models, Gemini Nano
 - Ética em IA mobile

Leituras obrigatórias (pré-Aula 6)

- Newman, S. — *Building Microservices*, cap 4 (BFF)
- Porcello & Banks — *Learning GraphQL* (O'Reilly), cap 4–6
- OWASP — *MASVS v2.1* (resumo guiado)
- IETF RFC 7636 (PKCE) e RFC 8252 (OAuth Native Apps)
- MLC-LLM project — docs (mlc.ai) — preparação Aula 6

Este é conteúdo bônus (gravado)

Sem entrega, sem pontos — assista quando quiser antes da Aula 6.

Aula 6 — Performance, Observabilidade, IA on-Device & Apresentações Finais é a última aula ao vivo (ver Canvas pra data/prazos), com:

- Atividade 5 entregue
- Quiz IA respondido
- **Projeto Final** pronto pra apresentação (10min + Q&A 5min por grupo)

Obrigado!

 jackson.96@gmail.com

 linkedin.com/in/3jacksonsmith

 github.com/jacksonsmith

Conteúdo bônus — sem prazo, assista quando quiser